

A CPU-Centric Platform for Literature Mining and LLM Efficiency Studies: Design, Evaluation, and Engineering Lessons

Ayesha Rahman – Independent Researcher
ayesharahman7755@gmail.com

Abstract

This work presents a practical research platform for studying large language models (LLMs) under realistic compute constraints. The system targets CPU-only environments and integrates (i) automated literature collection and summarization from arXiv and Semantic Scholar, (ii) dense retrieval and retrieval-augmented generation (RAG) over scientific papers, (iii) exploratory data analysis and survey-cleaning utilities, and (iv) an efficiency toolkit for profiling latency, throughput, batching, quantization, LoRA-style parameter-efficient fine-tuning, and KV-cache behavior. All components are implemented in Python using HuggingFace Transformers [Wolf et al., 2020], PyTorch, and scikit-learn, exposed through a Streamlit interface, and orchestrated via a Makefile and a CPU-only Dockerfile for reproducible experiments.

From an end-to-end run, the platform yields summarization quality (ROUGE-L 0.18, BERTScore-F1 0.86), dense-retrieval effectiveness (nDCG@5 $\approx$ 0.85), and detailed efficiency metrics such as latency vs. batch size, tokens-per-second vs. prompt length, and CPU-only KV-cache micro-benchmarks. The platform is designed as a testbed for research on LLM efficiency, model optimization, and modular NLP systems, and provides a concrete example of how small instruction-tuned models (MiniLM, FLAN-T5-small [Raffel et al., 2020]) can support non-trivial research workflows without access to GPUs.

1 Introduction

Large language models (LLMs) have rapidly evolved into general-purpose research tools, supporting tasks such as literature triage, summarization, code synthesis, and exploratory data analysis. Their ability to ingest large corpora and produce structured outputs has made them central to empirical machine learning, data-centric science, and applied AI. However, much of the recent work on LLM tooling and infrastructure assumes GPU-rich environments and does not address practical constraints faced by many institutions, such as CPU-only servers or modest cloud budgets.

At the same time, the software ecosystem around LLMs is fragmented. Tools for literature retrieval, RAG, model evaluation, and efficiency profiling are often developed in isolation, making it difficult to run controlled experiments that span the full stack from data ingestion to deployment-style metrics. For researchers interested in efficiency, modularity, or performance-constrained settings (e.g., edge devices or shared HPC clusters), it is useful to have an integrated, reproducible platform that combines *end-to-end functionality* with *fine-grained measurement*.

This paper introduces such a platform: a CPU-centric LLM research assistant for automated literature review, RAG, and efficiency evaluation. Rather than proposing a new model architecture, the work focuses on engineering and empirical questions:

- How far can relatively small models (MiniLM, FLAN-T5-small) be pushed in terms of retrieval and summarization quality when combined with classical IR and RAG techniques?

- What are the latency and throughput characteristics of these models under realistic CPU-only constraints, as a function of prompt length, batch size, and quantization settings?
- How do simple parameter-efficient fine-tuning (PEFT) and KV-cache configurations behave in this environment, and what trade-offs arise compared to common GPU-centric expectations?

To address these questions, an end-to-end pipeline is designed and implemented that: (i) ingests, normalizes, and clusters scientific literature; (ii) builds dense indices and RAG-style QA over the resulting corpus; (iii) provides data-analysis and survey-cleaning utilities for tabular datasets; and (iv) exposes a suite of efficiency experiments via a unified interface.

Contributions. The main contributions of this work are:

- **A reproducible, CPU-friendly LLM stack** that integrates literature mining, summarization, RAG, tabular analysis, and survey cleaning into a single modular codebase.
- **An efficiency evaluation toolkit** for CPU-only environments, covering latency, tokens-per-second, batching behavior, quantization stubs, KV-cache micro-benchmarks, and LoRA-based PEFT.
- **Empirical characterization** of how a small FLAN-T5 model and MiniLM embeddings perform on literature summarization and dense retrieval, including quality metrics (ROUGE, BERTScore) and ranking metrics (nDCG@5).
- **An extensible engineering blueprint**—implemented with PyTorch, HuggingFace, and Streamlit—that can be used for further research on efficient LLM architectures, modular pipelines, and multimodal extensions.

2 Background

The platform draws on several established lines of research: transformer architectures, sentence-level embeddings, information retrieval, summarization, and parameter-efficient adaptation.

2.1 Transformer Models and Text-to-Text Frameworks

Transformer architectures [Vaswani et al., 2017] underpin most modern LLMs. The T5 framework [Raffel et al., 2020] casts NLP tasks into a unified text-to-text format, enabling a single encoder-decoder model to support translation, summarization, classification, and QA. FLAN-T5 [Chung et al., 2022] extends T5 with large-scale instruction tuning, improving zero-shot and few-shot capabilities while maintaining relatively small model variants suitable for CPUs. In this platform, FLAN-T5-small is used as a summarization and generation backbone to balance quality and efficiency.

2.2 Sentence Embeddings and Dense Retrieval

Dense vector representations have become standard for semantic retrieval. Sentence-BERT [Reimers and Gurevych, 2019] demonstrated that fine-tuned transformer encoders can produce high-quality sentence embeddings, while FAISS [Johnson et al., 2019] and related libraries provide scalable approximate nearest-neighbor search. The MiniLM-based model `sentence-transformers/all-MiniLM-L6-v2` is adopted as a lightweight encoder that offers a favorable trade-off between semantic quality and CPU performance.

2.3 Summarization and Evaluation Metrics

Abstractive summarization is commonly evaluated using ROUGE [Lin, 2004], which measures n-gram overlap, and BERTScore [Zhang et al., 2020], which computes similarity in an embedding space. These metrics are not perfect correlates of human judgment, but they are widely used and easy to integrate into automated pipelines. In this platform they are used to assess the quality of literature summaries and to approximate factual alignment between RAG answers and their supporting contexts.

2.4 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [Lewis et al., 2020] couples dense retrieval with generative models, improving factuality and enabling open-domain QA without retraining large LMs. Production systems often rely on GPU-accelerated encoders, cross-encoders for reranking, and complex orchestration. In contrast, the implementation here focuses on a CPU-safe variant that combines MiniLM-based retrieval, optional TF-IDF reranking, and simple factuality diagnostics while keeping the code modular and inspectable.

2.5 Parameter-Efficient Fine-Tuning

Full fine-tuning of LLMs is expensive in both compute and memory. Methods such as LoRA [Hu et al., 2022] introduce low-rank adapters into attention projections, dramatically reducing the number of trainable parameters while preserving or improving task performance. A LoRA-based training and evaluation loop is integrated into the platform to explore how much improvement can be obtained from PEFT in CPU-only settings.

3 System Overview

The codebase is organized into modular Python packages that mirror the main functional components:

- **src/literature**: literature fetching, normalization, embedding, clustering, summarization, and reporting.
- **src/data_analysis**: exploratory data analysis (EDA), statistical tests, visualization, and LLM-generated commentary (tested via `tests/test_analyze_data.py`).
- **src/rag**: dense index construction, RAG QA, hybrid reranking, multi-query expansion, and retrieval evaluation.
- **src/llm_opt**: quantization, efficiency metrics, KV-cache benchmarking, PEFT training, and profiling.
- **src/surveys**: survey cleaning utilities for Likert and multi-choice data.
- **src/reporting**: report and table generation (Markdown, PDF, Excel).
- **ui/app.py**: a Streamlit UI that orchestrates the above modules and exposes configuration options.

Configuration is centralized in `config.yaml` and can be overridden via environment variables, allowing the UI to select summarization models, maximum tokens, and RAG parameters. A Makefile defines composite targets such as `make literature`, `make rag`, `make llmopt`, and `make report`, providing a simple orchestration layer. A CPU-only Dockerfile (`Dockerfile.cpu`) packages all dependencies, enabling reproducible runs on commodity servers.

The system is designed to be:

- **CPU-safe:** models default to small backbones, and GPU-specific code paths are guarded by availability checks.
- **Network-tolerant:** external APIs (arXiv, Semantic Scholar, CrossRef) implement retry logic and backoff to handle rate limits.
- **Tested:** unit tests validate parsing, metrics, and end-to-end flows for core components.

The following sections describe the main modules in more detail and summarize the empirical behavior observed in an end-to-end run.

4 Literature Mining and Summarization

4.1 Data Ingestion and Normalization

The literature pipeline begins by querying arXiv and Semantic Scholar for a configurable search phrase (default: “LLM efficiency”):

- `fetch_papers.py` queries the arXiv API, handles HTTP 429 via exponential backoff, and caps the total number of returned items.
- `fetch_semantic_scholar.py` calls the Semantic Scholar Graph API, optionally using an API key for higher rate limits and requesting metadata such as titles, abstracts, venues, DOIs, and URLs.

`normalize_papers.py` merges these sources into a unified JSON list `data/literature_items.json`. It parses arXiv XML, normalizes whitespace, extracts IDs and summaries, and aligns Semantic Scholar entries via fields such as `paperId` and external IDs. Deduplication is performed by case-folded title.

An optional enrichment step, `fetch_crossref.py`, queries the CrossRef API for missing DOIs and URLs based on title search, improving downstream citation quality.

4.2 Embedding and Clustering

To build semantic structure over the corpus, `embed_cluster.py` uses the MiniLM encoder `sentence-transformers/all-MiniLM-L6-v2` [Reimers and Gurevych, 2019]. Each item is represented as

input = title — summary,

tokenized with `AutoTokenizer` and encoded with `AutoModel`. The implementation mean-pools the last hidden layer using the attention mask and L2-normalizes the resulting vectors, following standard sentence-transformer practice while avoiding an explicit dependency on the `sentence-transformers` package.

The embeddings are clustered with KMeans (`sklearn.cluster.KMeans`) using $k = \min(6, N)$, where N is the number of papers. A 2D PCA projection is saved as `plots/literature_pca.npy` and plotted in `plot_clusters.py`. When `umap-learn` is installed, `umap_projection.py` generates an additional UMAP visualization.

4.3 Summarization with FLAN-T5

Paper-level abstractive summaries are generated in `summarize_papers.py` using `google/flan-t5-small` [Raffel et al., 2020, Chung et al., 2022]. The script:

1. Loads `data/literature_items.json`.
2. Constructs inputs as “`title. summary`” for up to 20 items (for quick runs).
3. Performs batched generation with `AutoModelForSeq2SeqLM.generate`, using beam search (4 beams), a context max length of 1024, and configurable `max_new_tokens`.
4. Writes `results/summaries.json` containing titles, abstracts, and generated summaries.

Timing information is stored in `results/metrics.json`. On a 16-core CPU-only machine, summarizing 20 items with FLAN-T5-small completes in roughly 90 seconds.

4.4 Cluster-Level Summaries and Auto Review

`cluster_summaries.py` groups papers by KMeans cluster and prompts the same model to produce higher-level thematic summaries. For each cluster, it constructs a bullet list of up to 12 “`title: summary`” entries and requests a 6–8 sentence synthesis that explicitly avoids unsupported claims. The resulting summaries are combined in `auto_review_report.py` to form `reports/auto_review.md`, which organizes the corpus by cluster and provides within-cluster paper lists.

4.5 Citation Link Injection and BibTeX Export

`cite_link_injection.py` cross-references titles in `results/summaries.json` with `data/literature_items.json` to attach DOIs and URLs to summaries or existing Markdown documents. `save_bibtex.py` exports a BibTeX file (`reports/refs.bib`), using slugified titles as keys and annotating entries as auto-generated.

4.6 Summarization Evaluation

`src/eval/eval_summaries.py` computes ROUGE-L and BERTScore-F1 between generated summaries and original abstracts using Lin [2004] and Zhang et al. [2020]. The metrics are stored in `results/summary_metrics.json`. In the representative run, the following values are obtained:

- ROUGE-L F1: 0.1837
- BERTScore F1: 0.8571

These values indicate moderate lexical overlap but high semantic similarity, consistent with expectations for a small instruction-tuned model.

5 Tabular Data Analysis and Survey Cleaning

5.1 Arrow-Safe CSV Ingestion and EDA

The data-analysis path is implemented in `src/data_analysis/analyze_data.py` and surfaced in the Streamlit UI. CSV files are loaded into pandas with robust dtype handling and missing-value detection. The helper `load_csv_arrow_safe` normalizes boolean tokens, coerces numeric columns, parses dates, and produces Arrow-compatible dtypes for reliable downstream visualization.

An end-to-end test constructs a synthetic CSV with two numeric features, a group column, and a binary target. The pipeline computes descriptive statistics, missing-value analysis, optional simple models, and a summary JSON. In the real dataset contained in `results/data_analysis_summary.json`, the assistant analyzes a 50-row table with:

- 4 numeric columns and 1 additional column,
- an inferred clustering with $k = 3$ (via KMeans),
- generated artifacts such as correlation matrices and histograms.

Missing values are summarized in `results/missing_values.csv`, which is enforced by the test suite. Additional scripts such as `visualize_results.py` create PNG figures under `results/visualizations/`.

5.2 LLM Commentary on Data

`src/data_analysis/llm_commentary.py` reads numeric summaries and prompts an LLM to produce short narrative commentary. The text is written to `results/data_analysis_commentary.md` and later embedded into the final report, providing a lightweight example of combining statistical summaries with generative explanations.

5.3 Survey Cleaning Utilities

`src/surveys/survey_clean.py` offers reusable utilities for survey-style CSVs:

- Configurable Likert mappings (e.g., “strongly disagree:1, ..., strongly agree:5”) applied to specified columns.
- Multi-choice questions (stored as delimited strings) split and one-hot encoded into indicator columns with an accompanying codebook.

On the provided survey CSV, the cleaner generates `results/survey_clean_log.json` summarizing row counts, mappings, and option vocabularies. The cleaned data is stored in `data/uploads/survey_cleaned.csv` for downstream analysis.

6 Retrieval-Augmented Generation (RAG)

6.1 Dense Index Construction

`src/rag/build_index.py` converts `data/literature_items.json` into an embedding matrix `results/rag/doc_vecs.npy` using the MiniLM encoder from Section 4. Metadata is saved in

`results/rag/items.json`. If FAISS is available, a flat inner-product index is constructed and stored as `results/rag/index.faiss`; otherwise, similarity search falls back to NumPy.

`results/rag/meta.json` records the index backend and number of documents, enabling consistent evaluation across runs.

6.2 RAG QA and Factuality Check

`rag_qa.py` implements a simple RAG pipeline:

1. Encode the user question with MiniLM.
2. Retrieve top- k documents via FAISS or NumPy cosine similarity.
3. Concatenate titles and abstracts into a context block.
4. Prompt FLAN-T5-small to answer using only this context, discouraging unsupported claims.

The output `results/rag_qa.json` contains the question, answer, and supporting citations with similarity scores. For the query

“What are current methods to speed up LLM inference?”

the system returns an answer plus five cited papers.

`factuality_check.py` assesses alignment by concatenating the summaries of the cited documents into a pseudo-reference and computing ROUGE-L F1 against the generated answer. In the sample run, ROUGE-L F1 is 0.2439, slightly above a configurable faithfulness threshold.

6.3 Retrieval Evaluation Metrics

`rag_eval_metrics.py` evaluates retrieval quality independently of generation. Given `doc_vecs.npy` and queries from `results/rag_eval.csv`, it:

- Encodes each query with MiniLM.
- Retrieves top-5 documents.
- Computes relevance via token overlap between query and document text.
- Computes nDCG and a diversity metric (average pairwise cosine distance among top-5 embeddings).

For two representative queries:

1. “What are current methods to speed up LLM inference?”
2. “How do diffusion LLMs differ from autoregressive ones?”

the system reports:

- nDCG@5: 0.8499 and 0.8710.
- Diversity: 0.4287 and 0.4152.

These results suggest that even a small encoder can yield reasonably effective dense retrieval over this domain.

6.4 Hybrid Reranking and Multi-Query Expansion

`hybrid_reranker.py` explores a lightweight hybrid strategy combining:

- Dense similarity from MiniLM embeddings (cosine similarity).
- Sparse similarity from a scikit-learn `TfidfVectorizer`.

Scores are linearly combined with mixture parameter α (default 0.6). For the query “Methods to speed up LLM inference”, `results/rag/hybrid_search_demo.json` logs re-ranked candidates and their hybrid scores.

`multiquery_expansion.py` implements handcrafted query expansion based on seed phrases (e.g., “speed up”, “throughput”, “quantization”) and generic rewrites. The expansions, stored in `results/rag_multiquery.json`, can be used for multi-vector retrieval or analysis of query diversity.

7 LLM Efficiency and Optimization Toolkit

7.1 Quantization and Model Variants

`quantize_models.py` loads the base summarization model and optionally applies:

- BitsAndBytes 8-bit or 4-bit quantization (when CUDA and `bitsandbytes` are available).
- CPU dynamic quantization for linear layers when GPUs are unavailable.

Model variants are stored under the directory `experiments/quantized/` with subdirectories `fp32`, `fp16`, `int8-bnb`, `int4-bnb`, and `int8-cpu`. In the current run, metrics are available for the `fp32` variant, but the code paths for additional variants are in place for future experiments.

7.2 Latency and VRAM Metrics

`evaluate_efficiency.py` benchmarks each available model:

- Runs generation on a small set of prompts.
- Records average latency per request.
- When CUDA is present, logs VRAM usage via `torch.cuda.memory_reserved`.

Metrics are written to `results/efficiency_metrics.csv`. For the `fp32` CPU model, the average latency is 1.735 s per prompt, with VRAM reported as 0 MB.

`compare_results.py` merges efficiency metrics with summarization quality metrics from `summary_metrics.json`, yielding `results/benchmark_table.csv`. This table supports trade-off visualizations via `plot_tradeoffs.py`, which outputs `plots/tradeoff_curves.png`.

7.3 Tokens-per-Second and Batch Ablations

`eval_tokens_per_sec.py` measures throughput as a function of prompt length for the `fp32` model, generating up to 200 tokens. The recorded results (`results/tokens_per_sec.json`) include:

Prompt tokens	Generated	Latency (s)	Tokens/s
32	200	14.19	12.60
128	200	8.66	23.11
512	200	11.49	17.42
1024	200	18.11	11.05

Throughput peaks around 128 input tokens, highlighting the trade-off between encoder cost and decoding efficiency.

`batch_ablation.py` examines the effect of batch size on latency and throughput for a fixed prompt. For batch sizes $\{1, 2, 4, 8, 16, 32\}$, the script logs metrics in `results/batch_ablation.json`, including:

Batch size	Latency (s)	Requests/s
1	0.552	1.81
2	0.847	2.36
4	1.516	2.64
8	3.231	2.48
16	6.239	2.56
32	10.788	2.97

Even without GPUs, batching significantly improves throughput, at the cost of increased per-request latency.

7.4 KV Cache Micro-Benchmark

`kv_cache_bench.py` evaluates KV-cache behavior using `sshleifer/tiny-gpt2`. For short sequences on CPU, the recorded times (`results/kv_cache_bench.json`) are:

- Time with cache: 1.861 s,
- Time without cache: 0.371 s,
- Speedup ratio (no cache / cache): 0.20.

Contrary to common GPU-focused intuition, enabling the KV cache can *slow down* small models on CPU for short sequences due to cache management overhead. This illustrates the importance of environment-specific efficiency studies.

7.5 System Resources and Environment

`system_resource_report.py` gathers basic hardware statistics. In the reference environment (`results/system_resources.json`):

- `cpu_count`: 16,
- RAM: approximately 16 GB (15.38 GB total, 8.10 GB available),
- No GPU is detected (empty `nvidia-smi` output).

These constraints motivate the CPU-centric design and the choice of relatively small models.

7.6 PEFT Training and Evaluation

`train_peft.py` implements LoRA-based fine-tuning using the PEFT library [Hu et al., 2022]. The pipeline:

- Uses literature items as training data, pairing abstract+title as input and FLAN-T5 summaries as targets.
- Attaches LoRA modules to attention projections (`q`, `k`, `v`, `o`) with rank 8 and $\alpha = 16$.
- Trains for three epochs with an 80/20 train-validation split and AdamW optimization.
- Saves the best checkpoint under `experiments/peft_flan_t5/checkpoint-best`.

`peft_eval.py` evaluates the checkpoint against the same summarization targets, computing ROUGE-L and BERTScore-F1 and writing results to `results/peft_eval.json`. `peft_export.py` merges LoRA weights into the base model to produce a standalone artifact.

7.7 TorchScript Export and Multimodal Stub

`triton_export.py` traces a tiny GPT-2 causal LM into TorchScript, yielding `results/triton/model.ts` and `results/tiny_gpt2_traced.pt` as minimal examples of serving-oriented export.

`multimodal/llava_stub.py` currently acts as a placeholder for future multimodal integration, producing a stub JSON description of an image. This is intended as the insertion point for LLaVA-style [Liu et al., 2023] models or other multimodal backbones.

8 User Interface and Reporting

8.1 Streamlit Application

`ui/app.py` wraps the system in a Streamlit UI with tabs for metrics, summaries, clusters, data, RAG, PEFT, efficiency, and surveys. The app:

- Exposes text inputs for model name, maximum tokens, batch size, and RAG top- k .
- Forwards configuration via environment variables to underlying scripts.
- Wraps command-line invocations in a helper `run_cmd` that captures stdout/stderr and displays logs.
- Provides file uploaders for CSVs and survey data, previewing them with Arrow-safe dtypes.
- Visualizes PCA cluster plots, RAG outputs and factuality checks, PEFT metrics, and efficiency figures.

This interface demonstrates how complex research workflows can be made accessible to non-expert users while retaining full reproducibility for experiments.

8.2 Final Report and Tables

`reporting/report_generator.py` builds a final Markdown report (`reports/final_report.md`) and PDF (`reports/final_report.pdf`) using ReportLab. The report includes:

- An abstract summarizing summarization and retrieval metrics.
- A literature review section embedding the auto-generated cluster summaries.
- A data-analysis section embedding the LLM commentary.
- An efficiency section referencing the benchmark tables and trade-off curves.

`insert_figures.py` optionally augments reports with figure inclusions for cluster plots, correlation heatmaps, trade-off curves, and UMAP projections.

`table_export.py` collates CSV and JSON metrics into an Excel workbook (`reports/summary_tables.xlsx`), creating one sheet per dataset (efficiency metrics, benchmark tables, RAG evaluations, tokens-per-second, etc.), using `pandas.json_normalize` for nested JSON.

9 Results and Discussion

Across the end-to-end run represented in the `results` folder, the system demonstrates:

- **Summarization quality:** ROUGE-L 0.1837 and BERTScore-F1 0.8571 on 20 literature items with FLAN-T5-small.
- **Retrieval quality:** nDCG@5 in the range 0.85–0.87 with moderate diversity (0.41–0.43) for two LLM-related queries using MiniLM embeddings.
- **Efficiency:** For fp32 summarization, average latency is 1.74 s per prompt; tokens-per-second peak at ≈ 23 for medium-length prompts; batching can nearly triple throughput with acceptable latency trade-offs.
- **Data analysis and survey cleaning:** A 50-row dataset is processed with automatic statistics, clustering ($k = 3$), visualizations, and survey transformations, all orchestrated from a single UI.

Limitations include the use of relatively small models (FLAN-T5-small, MiniLM), a simple RAG pipeline without cross-encoder reranking, and efficiency benchmarks that are illustrative rather than exhaustive. Nonetheless, the platform provides a concrete baseline for research on LLM efficiency, modularity, and deployment in constrained environments.

10 Related Work

Transformers and Pretrained LMs. This system builds on the Transformer family [Vaswani et al., 2017] and the T5 text-to-text framework [Raffel et al., 2020], as implemented in HuggingFace Transformers [Wolf et al., 2020]. FLAN-T5 [Chung et al., 2022] informs the choice of instruction-tuned backbones.

Sentence Embeddings and Dense Retrieval. The use of MiniLM encoders is inspired by Sentence-BERT [Reimers and Gurevych, 2019]. For scalable retrieval, the design relies conceptually on FAISS [Johnson et al., 2019], while providing a NumPy-based fallback for CPU-only setups.

Retrieval-Augmented Generation. The RAG component follows the general paradigm of retrieval-augmented generation [Lewis et al., 2020], adapted here for a small monolingual encoder and a single generative model.

Evaluation Metrics. Summarization and factuality proxies are measured with ROUGE [Lin, 2004] and BERTScore [Zhang et al., 2020].

Parameter-Efficient Fine-Tuning. The PEFT module implements the LoRA method [Hu et al., 2022], which has become a standard technique for efficient adaptation of large models.

Multimodal LLMs. Although the multimodal integration is currently a stub, future work is informed by multimodal instruction-tuned models such as LLaVA [Liu et al., 2023].

11 Conclusion and Future Work

This paper has described a CPU-centric platform that integrates literature mining, RAG, tabular data analysis, and an LLM efficiency toolkit into a single, reproducible stack. The implementation combines modern NLP components (Transformers, sentence embeddings, RAG, LoRA) with pragmatic engineering practices (Docker, Makefile orchestration, a Streamlit UI, and a pytest suite).

There are several natural directions for future work:

- **Stronger models and comparative studies.** Integrate more capable models (e.g., larger FLAN-T5 variants or open-source LLaMA-family models) and run systematic comparisons of quality, latency, and memory on CPU vs. GPU.
- **Richer RAG pipelines.** Extend the RAG component with learned rerankers, improved factuality metrics, multi-vector retrieval, and support for domain-specific indices (e.g. code repositories or institutional paper collections).
- **Multimodal extensions.** Replace the current LLaVA-style stub with a working vision-language path for tasks such as figure captioning and chart explanation, following multimodal LLM work [Liu et al., 2023].
- **Advanced data and survey analytics.** Add richer statistical modeling (e.g., mixed-effects models) and causal inference utilities to the tabular analysis module, and integrate LLM-based explanation for more complex analyses.
- **Broader efficiency experiments.** Run the same pipeline across multiple hardware settings—from laptops to HPC nodes and edge devices—to systematically map the trade-offs between quality, latency, energy use, and cost.

Overall, the platform serves as both a practical research assistant and a testbed for studying efficient LLM deployment in resource-constrained environments, and can be extended to support more advanced models, multimodal capabilities, and larger-scale experiments.

References

- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, et al. Scaling instruction-finetuned language models. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2210.11416>. arXiv:2210.11416.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. doi: 10.1109/TBDATA.2019.2921572. URL <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pages 74–81, Barcelona, Spain, 2004. Association for Computational Linguistics. URL <https://aclanthology.org/W04-1013>.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. LLaVA: Large language and vision assistant, 2023. URL <https://arxiv.org/abs/2304.08485>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1410. URL <https://aclanthology.org/D19-1410>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020. doi: 10.18653/v1/2020.emnlp-demos.6. URL <https://aclanthology.org/2020.emnlp-demos.6>.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SkeHuCVFDr>.